

## Experiment 03 - Classification algorithm

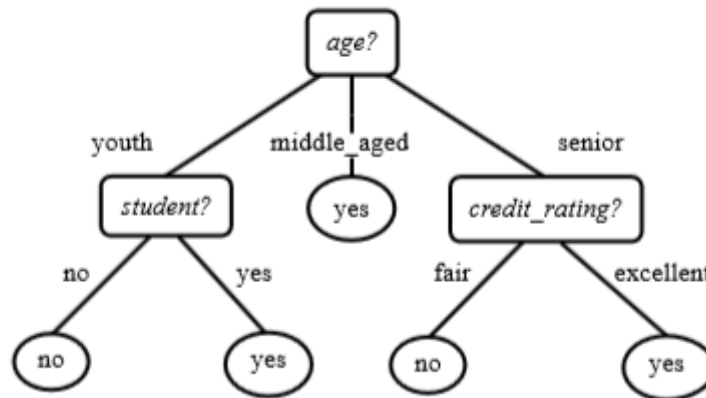
**Learning Objective:** Student should be able to Implement Classification algorithm (Decision Tree)

**Tools:** Java/C/C++/Python/Data Mining Tool

**Theory:** Implement Decision Tree using ID3/C4.5

### **Decision Tree:**

A decision tree is a flowchart like tree structure, where each internal node(non-leafnode) denotes a test on an attribute, each branch represents an outcome of the test, and each leaf node (or terminal node) holds a class label. The topmost node in a tree is the root node.



A typical decision tree is shown in above figure. It represents the concept buys computer, that is, it predicts whether a customer at AllElectronics is likely to purchase a computer. Internal nodes are denoted by rectangles, and leaf nodes are denoted by ovals. Some decision tree algorithms produce only binary trees (where each internal node branches to exactly two other nodes), whereas others can produce nonbinary trees. “How are decision trees used for classification?” Given a tuple, X, for which the associated class label is unknown, the attribute values of the tuple are tested against the decision tree. A path is traced from the root to a leaf node, which holds the class prediction for that tuple. Decision trees can easily be converted to classification rules. “Why are decision tree classifiers so popular?” The construction of decision tree classifiers does not require any domain knowledge or parameter setting, and therefore is appropriate for exploratory knowledge discovery. Decision trees can handle high dimensional data. Their representation of acquired knowledge in tree form is intuitive and generally easy to assimilate by humans. The learning and classification steps of decision tree induction are simple and fast. In general, decision tree classifiers have good accuracy. However, successful use may depend on the data at hand. Decision tree induction algorithms have been used for classification in many application areas, such as medicine, manufacturing and production, financial analysis, astronomy, and molecular biology. Decision trees are the basis of several commercial rule induction systems.

## Result and Discussion:

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>import math  # 1. Dataset: AllElectronics "buys_computer" dataset dataset = [     {'age': 'youth', 'student': 'no', 'credit_rating': 'fair', 'buys_computer': 'no'},     {'age': 'youth', 'student': 'no', 'credit_rating': 'excellent', 'buys_computer': 'no'},     {'age': 'middle_aged', 'student': 'no', 'credit_rating': 'fair', 'buys_computer': 'yes'},     {'age': 'senior', 'student': 'no', 'credit_rating': 'fair', 'buys_computer': 'yes'},     {'age': 'senior', 'student': 'yes', 'credit_rating': 'fair', 'buys_computer': 'yes'},     {'age': 'senior', 'student': 'yes', 'credit_rating': 'excellent', 'buys_computer': 'no'},     {'age': 'middle_aged', 'student': 'yes', 'credit_rating': 'excellent', 'buys_computer': 'yes'},     {'age': 'youth', 'student': 'no', 'credit_rating': 'fair', 'buys_computer': 'no'},     {'age': 'youth', 'student': 'yes', 'credit_rating': 'fair', 'buys_computer': 'yes'},     {'age': 'senior', 'student': 'yes', 'credit_rating': 'fair', 'buys_computer': 'yes'},     {'age': 'youth', 'student': 'yes', 'credit_rating': 'excellent', 'buys_computer': 'yes'},     {'age': 'middle_aged', 'student': 'no', 'credit_rating': 'excellent', 'buys_computer': 'yes'},     {'age': 'middle_aged', 'student': 'yes', 'credit_rating': 'fair', 'buys_computer': 'yes'},     {'age': 'senior', 'student': 'no', 'credit_rating': 'excellent', 'buys_computer': 'no'} ]  # 2. Function to calculate Entropy of a dataset def calculate_entropy(data):     total = len(data)     if total == 0:         return 0      counts = {}     for row in data:         label = row['buys_computer']         counts[label] = counts.get(label, 0) + 1</pre> | <pre>def build_tree(data, attributes):     labels = [row['buys_computer'] for row in data]      # Base Case 1: If all target outcomes are     identical     if labels.count(labels[0]) == len(labels):         return labels[0]      # Base Case 2: If no remaining features,     return majority class     if not attributes:         return max(set(labels), key=labels.count)      # Step A: Select feature with Maximum     Information Gain     gains = {}     for attr in attributes:         gains[attr] = calculate_info_gain(data, attr)     best_attr = max(gains, key=gains.get)      tree = {best_attr: {}}     remaining_attributes = [a for a in attributes if a != best_attr]      # Step B: Split dataset recursively     values = set(row[best_attr] for row in data)     for val in values:         subset = [row for row in data if row[best_attr] == val]         tree[best_attr][val] = build_tree(subset, remaining_attributes)      return tree  # 5. Function to predict class for a new tuple def predict(tree, sample):     if not isinstance(tree, dict):         return tree      root = list(tree.keys())[0]     val = sample.get(root)      if val in tree[root]:         return predict(tree[root][val], sample)     else:         return "Unknown"  # --- MAIN EXECUTION ---</pre> |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> entropy = 0 for count in counts.values():     p = count / total     entropy -= p * math.log2(p) return entropy  # 3. Function to calculate Information Gain for a specific attribute def calculate_info_gain(data, attribute):     total_entropy = calculate_entropy(data)      # Group data by attribute values     subsets = {}     for row in data:         val = row[attribute]         if val not in subsets:             subsets[val] = []         subsets[val].append(row)      # Calculate weighted entropy     weighted_entropy = 0     for subset in subsets.values():         prob = len(subset) / len(data)         weighted_entropy += prob * calculate_entropy(subset)      return total_entropy - weighted_entropy </pre> | <pre> attributes = ['age', 'student', 'credit_rating'] decision_tree = build_tree(dataset, attributes)  print("=" * 55) print("  EXPERIMENT 03: DECISION TREE CLASSIFIER (ID3)  ") print("=" * 55 + "\n")  print("1. GENERATED DECISION TREE STRUCTURE:") print(decision_tree)  print("\n2. TESTING CUSTOMER PREDICTIONS:") print("-" * 55)  # Test Sample 1: Youth, Student = Yes, Credit Rating = Fair test1 = {'age': 'youth', 'student': 'yes', 'credit_rating': 'fair'} print(f"Sample 1: {test1}") print(f"Prediction (Buys Computer?): {predict(decision_tree, test1)}\n")  # Test Sample 2: Senior, Credit Rating = Excellent test2 = {'age': 'senior', 'student': 'no', 'credit_rating': 'excellent'} print(f"Sample 2: {test2}") print(f"Prediction (Buys Computer?): {predict(decision_tree, test2)}")  print("=" * 55) </pre> |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## Result:

```

=====
EXPERIMENT 03: DECISION TREE CLASSIFIER (ID3)
=====

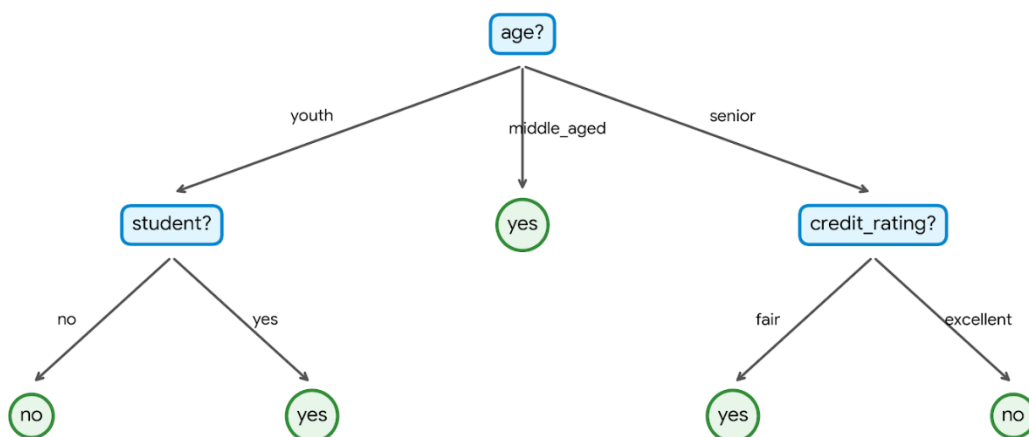
1. GENERATED DECISION TREE STRUCTURE:
{'age': {'senior': {'credit_rating': {'fair': 'yes', 'excellent': 'no'}}, 'youth': {'student': {'no': 'no',
'yes': 'yes'}}}, 'middle_aged': 'yes'}}

2. TESTING CUSTOMER PREDICTIONS:
-----
Sample 1: {'age': 'youth', 'student': 'yes', 'credit_rating': 'fair'}
Prediction (Buys Computer?): yes

Sample 2: {'age': 'senior', 'student': 'no', 'credit_rating': 'excellent'}
Prediction (Buys Computer?): no
=====

```

Decision Tree for 'buys\_computer'



**Learning Outcomes:** The student should have the ability to

LO1: compute Decision tree, Naïve Bayes

LO2: implement Decision tree, Naïve Bayes

LO3: demonstrate the outputs for Decision tree, Naïve Bayes

**Course Outcomes:** Upon completion of the course students will be able to Compare and evaluate different data mining techniques like classification.

**Conclusion:**

For Faculty Use

| Correction Parameters | Formative Assessment [40%] | Timely completion of Practical [ 40%] | Attendance / Learning Attitude [20%] |  |
|-----------------------|----------------------------|---------------------------------------|--------------------------------------|--|
| Marks Obtained        |                            |                                       |                                      |  |

## Experiment 04 - Linear Regression

**Learning Objective:** Student should be able to Implement Linear Regression

**Tools:** Java/C/C++/Python/Data Mining Tool

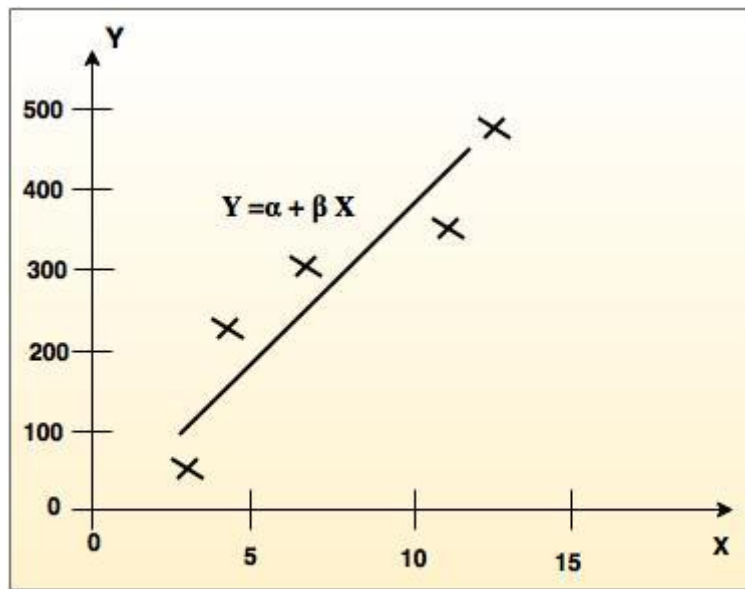
**Theory:** Implement of Linear Regression

### **Linear Regression:**

- It is simplest form of regression. Linear regression attempts to model the relationship between two variables by fitting a linear equation to observe the data.
- Linear regression attempts to find the mathematical relationship between variables.
- If outcome is straight line then it is considered as linear model and if it is curved line, then it is a non linear model.
- The relationship between dependent variable is given by straight line and it has only one independent variable.

$$Y = \alpha + \beta X$$

- Model 'Y', is a linear function of 'X'.
- The value of 'Y' increases or decreases in linear manner according to which the value of 'X' also changes.



**Linear Regression**

## Design:

# 1. Dataset based on the experiment graph (X: Independent Variable, Y: Dependent Variable)

# Example data: Experience (Years) vs Salary (in thousands)

X = [2.5, 4.0, 6.5, 11.0, 12.5]

Y = [55, 225, 300, 350, 475]

# 2. Function to compute Linear Regression coefficients (alpha and beta)

```
def train_linear_regression(X, Y):
```

```
    n = len(X)
```

```
    mean_x = sum(X) / n
```

```
    mean_y = sum(Y) / n
```

```
    numerator = sum((X[i] - mean_x) * (Y[i] - mean_y) for i in range(n))
```

```
    denominator = sum((X[i] - mean_x) ** 2 for i in range(n))
```

```
    beta = numerator / denominator
```

```
    alpha = mean_y - (beta * mean_x)
```

```
    return alpha, beta
```

# 3. Function to predict Y for a new given X value using formula  $Y = \alpha + \beta * X$

```
def predict(alpha, beta, x_val):
```

```
    return alpha + (beta * x_val)
```

# --- MAIN EXECUTION ---

```
alpha, beta = train_linear_regression(X, Y)
```

```
print("=" * 55)
```

```
print("    EXPERIMENT 04: LINEAR REGRESSION OUTPUT    ")
```

```
print("=" * 55 + "\n")
```

```
print(f"1. MODEL COEFFICIENTS:")
```

```
print(f" - Slope ( $\beta$ ) : {beta:.4f}")
```

```
print(f" - Intercept ( $\alpha$ ): {alpha:.4f}")
```

```
print(f" - Model Formula: Y = {alpha:.2f} + {beta:.2f} * X\n")
```

```
print("2. ORIGINAL DATA VS PREDICTED VALUES:")
```

```
print("-" * 55)
```

```
print(f"{'X (Input)':<12} | {'Y (Actual)':<12} | {'Y (Predicted)':<12}")
```

```
print("-" * 55)
```

```
for i in range(len(X)):
```

```
    y_pred = predict(alpha, beta, X[i])
```

```
    print(f"{'X[i]:<12} | {'Y[i]:<12} | {'y_pred:<12.2f}")
```

```
print("\n3. PREDICTION FOR NEW INPUT DATA:")
```

```
print("-" * 55)
```

```
new_x = 8.0
```

```
new_y = predict(alpha, beta, new_x)
```

```
print(f" Predicting Y for X = {new_x}:")
```

```
print(f" Resulting Y = {new_y:.2f}")
```

```
print("=" * 55)
```

## Result and Discussion:

```

Output

=====
EXPERIMENT 04: LINEAR REGRESSION OUTPUT
=====

1. MODEL COEFFICIENTS:
- Slope ( $\beta$ ) : 33.4462
- Intercept ( $\alpha$ ): 36.8426
- Model Formula:  $Y = 36.84 + 33.45 * X$ 

2. ORIGINAL DATA VS PREDICTED VALUES:
-----
X (Input) | Y (Actual) | Y (Predicted)
-----
2.5       | 55         | 120.46
4.0       | 225        | 170.63
6.5       | 300        | 254.24
11.0      | 350        | 404.75
12.5      | 475        | 454.92

3. PREDICTION FOR NEW INPUT DATA:
-----
Predicting Y for X = 8.0:
Resulting Y = 304.41
=====

=== Code Execution Successful ===
  
```

**Learning Outcomes:** The student should have the ability to

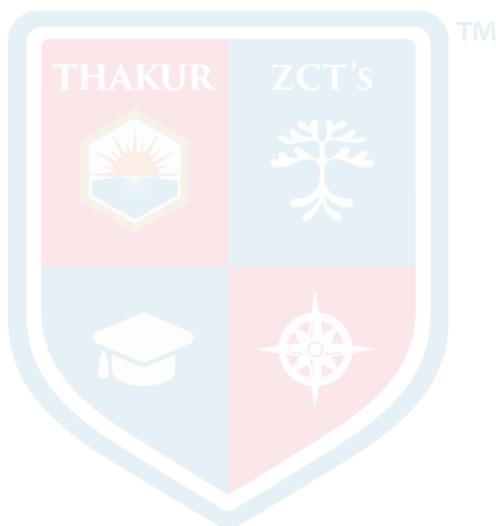
LO1: compute Linear Regression.

LO2: implement Linear Regression

LO3: demonstrate the outputs for Linear Regression.

**Course Outcomes:** Upon completion of the course students will be able to Compare and evaluate different data mining techniques like classification, prediction, clustering and association rule mining

**Conclusion:**



For Faculty Use

| Correction Parameters | Formative Assessment [40%] | Timely completion of Practical [ 40%] | Attendance / Learning Attitude [20%] |  |
|-----------------------|----------------------------|---------------------------------------|--------------------------------------|--|
| Marks Obtained        |                            |                                       |                                      |  |

## Experiment 06 – Hierarchical Clustering Algorithm

**Learning Objective:** Student should be able to Implement Clustering Algorithm (Hierarchical).

**Tools:** Java/C/C++/Python/Data Mining Tools

### **Theory: Implementation of Clustering Algorithm**

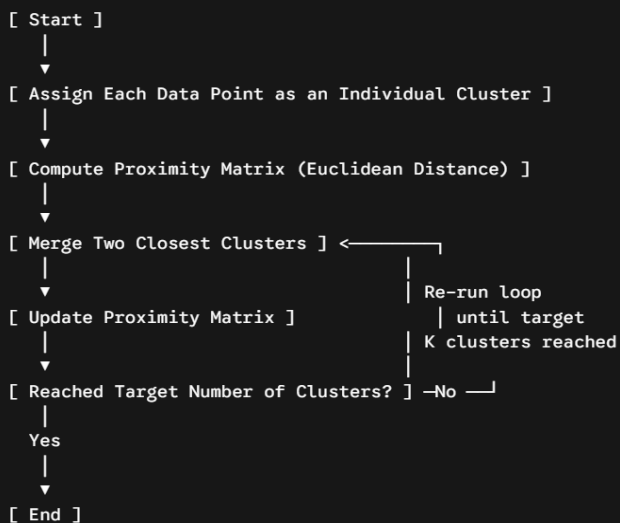
Cluster analysis or clustering is the assignment of a set of observations into subsets (called clusters) so that observations in the same cluster are similar in some sense. Clustering is a method of unsupervised learning, and a common technique for statistical data analysis used in many fields, including machine learning, data mining, pattern recognition, image analysis and bioinformatics.

### **Hierarchical Clustering:**

Hierarchical clustering is a method of cluster analysis which seeks to build a hierarchy of clusters without requiring pre-specifying the initial number of centroids. It works via two main approaches:

1. Agglomerative (Bottom-Up): Each observation starts in its own cluster, and pairs of clusters are merged iteratively as one moves up the hierarchy.
2. Divisive (Top-Down): All observations start in one cluster, and splits are performed recursively as one moves down the hierarchy.

### **ALGORITHM:**



### **Result and Discussion:**

```
import math

# 1. Dataset: [Age, Spending Score (1-100)]
data = [
    [18, 85], [20, 90], [22, 80], [19, 95], # Group 1: Young High-spenders
    [55, 20], [60, 15], [50, 25], [65, 10] # Group 2: Older Low-spenders
]

# Calculate Euclidean Distance between two 2D points
def euclidean_distance(p1, p2):
    return math.sqrt((p1[0] - p2[0])**2 + (p1[1] - p2[1])**2)

# Calculate Single Linkage Distance between two clusters (minimum distance)
def cluster_distance(cluster1, cluster2):
    min_dist = float('inf')
    for p1 in cluster1:
        for p2 in cluster2:
            dist = euclidean_distance(p1, p2)
            if dist < min_dist:
                min_dist = dist
    return min_dist

# Agglomerative Hierarchical Clustering Algorithm
def hierarchical_clustering(data, target_k):
    # Step 1: Assign each data point as its own individual cluster
    clusters = [[p] for p in data]

    # Step 2: Iteratively merge the closest clusters
    while len(clusters) > target_k:
        min_dist = float('inf')
        merge_pair = (0, 1)

        for i in range(len(clusters)):
            for j in range(i + 1, len(clusters)):
                dist = cluster_distance(clusters[i], clusters[j])
                if dist < min_dist:
                    min_dist = dist
                    merge_pair = (i, j)

        i, j = merge_pair
```

```
clusters[i].extend(clusters[j])
clusters.pop(j)

return clusters

# --- MAIN EXECUTION ---
target_clusters = 2
final_clusters = hierarchical_clustering(data, target_clusters)

# Display Results
print("=" * 55)
print(" PURE PYTHON HIERARCHICAL CLUSTERING (AGGLOMERATIVE) ")
print("=" * 55 + "\n")

for i, cluster in enumerate(final_clusters):
    print(f"--- CLUSTER {i + 1} ---")
    print("Data Points in Cluster:")
    for point in cluster:
        print(f"  -> Age: {point[0]}, Spending Score: {point[1]}")
    print()
```

### Output:

```
Output
=====
PURE PYTHON HIERARCHICAL CLUSTERING (AGGLOMERATIVE)
=====

--- CLUSTER 1 ---
Data Points in Cluster:
  -> Age: 18, Spending Score: 85
  -> Age: 20, Spending Score: 90
  -> Age: 19, Spending Score: 95
  -> Age: 22, Spending Score: 80

--- CLUSTER 2 ---
Data Points in Cluster:
  -> Age: 55, Spending Score: 20
  -> Age: 60, Spending Score: 15
  -> Age: 50, Spending Score: 25
  -> Age: 65, Spending Score: 10

=== Code Execution Successful ===
```

**Learning Outcomes:** The student should have the ability to

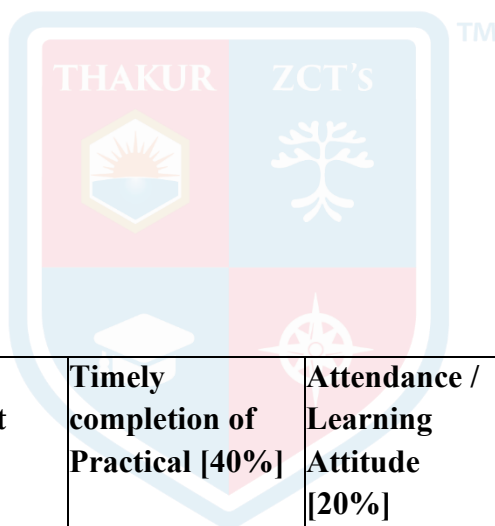
LO1: compute Clustering algorithm.

LO2: implement Clustering algorithm.

LO3: demonstrate the outputs for Clustering algorithm.

**Course Outcomes:** Upon completion of the course students will be able to Compare and evaluate different data mining techniques like clustering.

**Conclusion:**



For Faculty Use

| Correction Parameters | Formative Assessment [40%] | Timely completion of Practical [40%] | Attendance / Learning Attitude [20%] |  |
|-----------------------|----------------------------|--------------------------------------|--------------------------------------|--|
| Marks Obtained        |                            |                                      |                                      |  |

## Experiment 07 - Apriori Algorithm for Market Basket Analysis

**Learning Objective:** Student should be able to implement Apriori Algorithm for Market Basket Analysis

**Tools:** C/C++/Python/Data Mining Tool

**Theory:** Implementation of Apriori Algorithm for Market Basket Analysis

### *Apriori Algorithm:*

Apriori is a seminal algorithm proposed by R.Agrawal and R.Srikantin 1994 forming frequent itemsets for Boolean association rules. The name of the algorithm is based on the fact that the algorithm uses prior knowledge of frequent itemset properties, as we shall see following. Apriori employs an iterative approach known as a level-wise search, where  $k$ -itemsets are used to explore  $(k+1)$ -itemsets. First, the set of frequent 1-itemsets is found by scanning the database to accumulate the count for each item, and collecting those items that satisfy minimum support. The resulting set is denoted  $L_1$ . Next,  $L_1$  is used to find  $L_2$ , these to frequent 2-itemsets, which is used to find  $L_3$ , and so on, until no more frequent  $k$ -itemsets can be found. The finding of each  $L_k$  requires one full scan of the database. To improve the efficiency of the level-wise generation of frequent itemsets, an important property called the Apriori property, presented below, is used to reduce the search space. We will first describe this property, and then show an example illustrating its use.

Apriori property: All nonempty subsets of a frequent itemset must also be frequent.

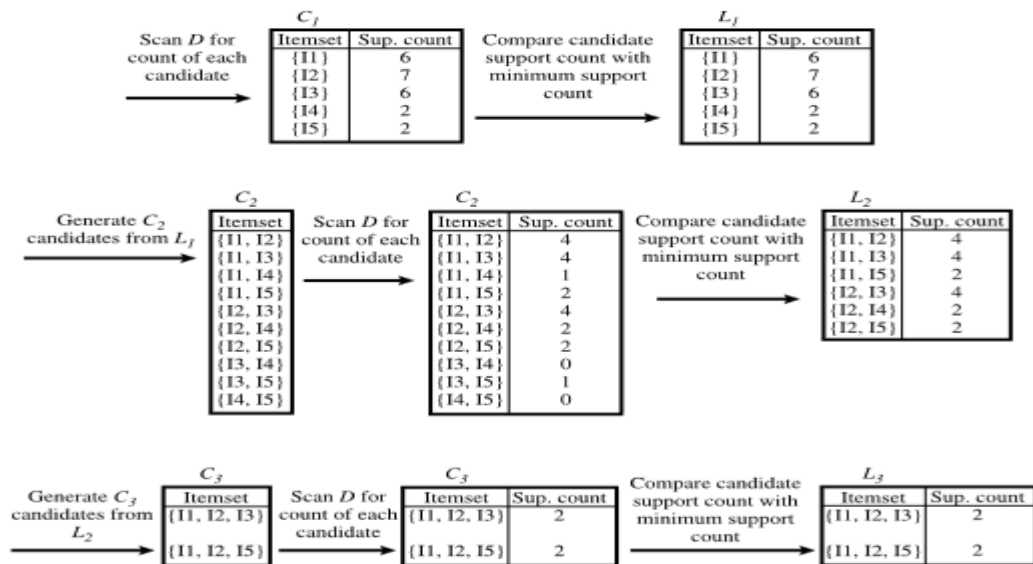
The Apriori property is based on the following observation. By definition, if an itemset  $I$  does not satisfy the minimum support threshold,  $\min \text{sup}$ , then  $I$  is not frequent; that is,  $P(I) < \min \text{sup}$ . If an item  $A$  is added to the itemset  $I$ , then the resulting itemset (i.e.,  $I \cup A$ ) cannot occur more frequently than  $I$ . Therefore,  $I \cup A$  is not frequent either; that is,  $P(I \cup A) < \min \text{sup}$ . This property belongs to a special category of properties called antimonotone in the sense that if a set cannot pass a test, all of its supersets will fail the same test as well. It is called antimonotone because the property is monotonic in the context of failing a test.<sup>7</sup> “How is the Apriori property used in the algorithm?” To understand this, let us look at how  $L_{k-1}$  is used to find  $L_k$  for  $k \geq 2$ .

A two-step process is followed, consisting of join and prune actions.

1. The join step:
2. The prune step:

Transactional data for an *AllElectronics* branch.

| TID  | List of item_IDs |
|------|------------------|
| T100 | I1, I2, I5       |
| T200 | I2, I4           |
| T300 | I2, I3           |
| T400 | I1, I2, I4       |
| T500 | I1, I3           |
| T600 | I2, I3           |
| T700 | I1, I3           |
| T800 | I1, I2, I3, I5   |
| T900 | I1, I2, I3       |



## Design:

```
from itertools import combinations
```

```
# 1. Dataset: Transactional data for AllElectronics (TID -> List of items)
```

```
transactions = [
    ['I1', 'I2', 'I5'], # T100
    ['I2', 'I4'],       # T200
    ['I2', 'I3'],       # T300
    ['I1', 'I2', 'I4'], # T400
    ['I1', 'I3'],       # T500
    ['I2', 'I3'],       # T600
    ['I1', 'I3'],       # T700
    ['I1', 'I2', 'I3', 'I5'], # T800
    ['I1', 'I2', 'I3']  # T900
]
```

```

min_support_count = 2 # Minimum support count threshold = 2

def get_support_count(candidate, transactions):
    """Calculates how many transactions contain the candidate itemset."""
    count = 0
    for transaction in transactions:
        if set(candidate).issubset(set(transaction)):
            count += 1
    return count

def apriori(transactions, min_sup):
    items = sorted(list(set(item for t in transactions for item in t)))
    C1 = [{item} for item in items]

    L = {}
    L1 = {}
    for candidate in C1:
        sup = get_support_count(candidate, transactions)
        if sup >= min_sup:
            L1[frozenset(candidate)] = sup

    L[1] = L1
    k = 2

    while True:
        prev_frequent = list(L[k-1].keys())
        if not prev_frequent:
            break

        C_k = set()
        for i in range(len(prev_frequent)):
            for j in range(i + 1, len(prev_frequent)):
                union_set = prev_frequent[i].union(prev_frequent[j])
                if len(union_set) == k:
                    # Prune Step: Check if all subsets of size k-1 are frequent
                    subsets = combinations(union_set, k - 1)
                    if all(frozenset(sub) in prev_frequent for sub in subsets):
                        C_k.add(union_set)

        L_k = {}
        for candidate in C_k:
            sup = get_support_count(candidate, transactions)
            if sup >= min_sup:
                L_k[frozenset(candidate)] = sup

        if not L_k:
            break

        L[k] = L_k
        k += 1

    return L

print("=" * 60)
print("  EXPERIMENT 07: APRIORI ALGORITHM FOR MARKET BASKET ANALYSIS  ")
print("=" * 60 + "\n")

frequent_itemsets = apriori(transactions, min_support_count)

```

```
for k, L_k in frequent_itemsets.items():
    print(f"--- FREQUENT {k}-ITEMSETS (L{k}) ---")
    print(f"{'Itemset':<25} | {'Support Count':<15}")
    print("-" * 45)
    for itemset, count in sorted(L_k.items(), key=lambda x: sorted(list(x[0]))):
        items_str = "{" + ",".join(sorted(list(itemset))) + "}"
        print(f"{'itemset':<25} | {'count':<15}")
    print()
print("=" * 60)
```

## Result and Discussion:

**Output**

```
=====
EXPERIMENT 07: APRIORI ALGORITHM FOR MARKET BASKET ANALYSIS
=====

--- FREQUENT 1-ITEMSETS (L1) ---
Itemset | Support Count
-----
{I1} | 6
{I2} | 7
{I3} | 6
{I4} | 2
{I5} | 2

--- FREQUENT 2-ITEMSETS (L2) ---
Itemset | Support Count
-----
{I1, I2} | 4
{I1, I3} | 4
{I1, I5} | 2
{I2, I3} | 4
{I2, I4} | 2
{I2, I5} | 2

--- FREQUENT 3-ITEMSETS (L3) ---
Itemset | Support Count
-----
{I1, I2, I3} | 2
{I1, I2, I5} | 2

=====
=== Code Execution Successful ===
```

**Learning Outcomes:** The student should have the ability to

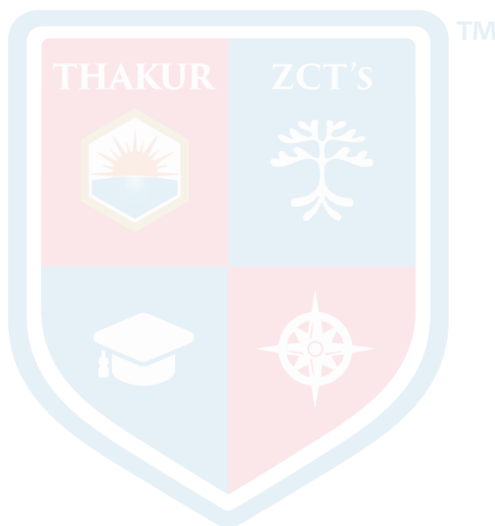
LO1: compute Apriori algorithm.

LO2: implement Apriori algorithm.

LO3: demonstrate the outputs for Apriori algorithm.

**Course Outcomes:** Upon completion of the course students will be able to demonstrate the Apriori algorithm.

**Conclusion:**



For Faculty Use

| Correction Parameters | Formative Assessment [40%] | Timely completion of Practical [ 40%] | Attendance / Learning Attitude [20%] |  |
|-----------------------|----------------------------|---------------------------------------|--------------------------------------|--|
| Marks Obtained        |                            |                                       |                                      |  |

## **Experiment 08 - Sales dashboard creation using Power BI**

**Learning Objective:** Student should be able to design and implement a Sales Dashboard using Power BI.

**Tools:** Microsoft Power BI Desktop, Sample Sales Dataset (Excel/CSV)

### **Theory:**

#### ***Sales Dashboard Creation in Power BI***

Power BI is a collection of software services, apps, and connectors that work together to turn unrelated sources of data into coherent, visually immersive, and interactive insights. A Sales Dashboard is a visual representation of sales data that provides an at-a-glance view of Key Performance Indicators (KPIs) relevant to a business's sales performance.

#### **Key Concepts in Power BI:**

1. Data Extraction and Transformation (ETL): Using Power Query to clean, reshape, and transform raw data.
2. Data Modeling: Creating relationships between different tables (e.g., linking a Sales table to a Date or Product table).
3. DAX (Data Analysis Expressions): A formula language used to create custom calculations, measures, and calculated columns.
4. Visualizations: Creating interactive charts, graphs, maps, and slicers to present the data.

#### **STEPS FOR CREATION:**

```

[ Start ]
  |
  v
[ Get Data (Import CSV/Excel Dataset) ]
  |
  v
[ Power Query (Clean & Transform Data) ]
  |
  v
[ Data Modeling (Establish Relationships) ]
  |
  v
[ Create Measures (Write DAX Formulas) ]
  |
  v
[ Build Visualizations (Charts, KPIs, Slicers) ]
  |
  v
[ Format Dashboard & Publish ]
  |
  v
[ End ]
  
```

### **Result and Discussion:**

For this experiment, a standard sample retail sales dataset was imported. The following DAX measures were created to analyze performance:

-- DAX Measures Created:

Total Gross Sales = SUM('financials'[Gross Sales])

Total Profit = SUM('financials'[Profit])

Total Units Sold = SUM('financials'[Units Sold])

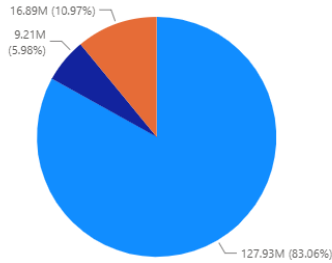
Total Discounts = SUM('financials'[Discounts])

Profit Margin % = DIVIDE([Total Profit], [Total Gross Sales], 0)

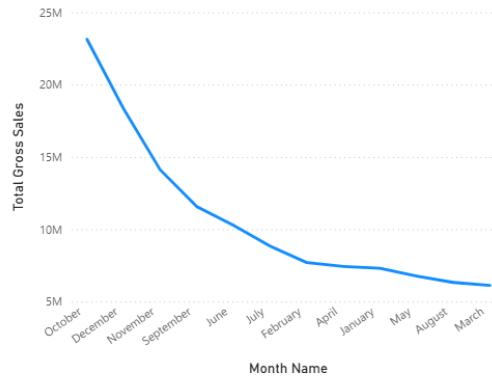
Dashboard Layout and Visuals Configured:

- Slicers (Filters): Year, Region, Product Category.
- KPI Cards: Displaying [Total Sales], [Total Profit], and [Total Orders].
- Line Chart: 'Total Sales by Month' to show trends over time.
- Clustered Bar Chart: 'Total Sales by Category' to compare product performance.
- Map Visual: 'Sales by City/State' to indicate geographical revenue hotspots.

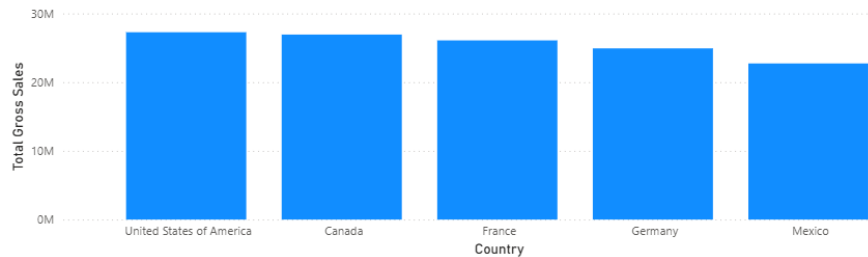
Total Gross Sales, Total Discounts and Total Profit



Total Gross Sales by Month Name

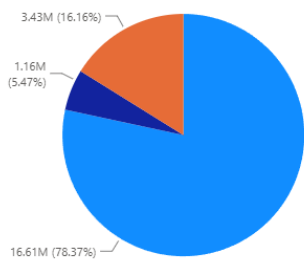


Total Gross Sales by Country

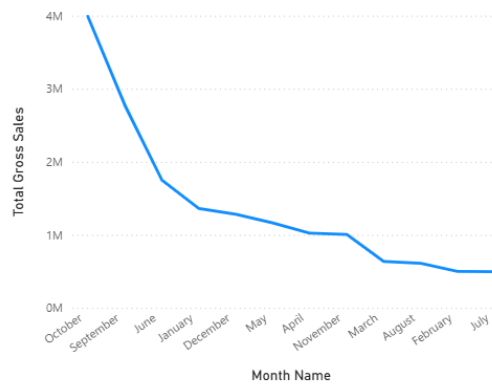


- Segment
- ☐ Channel Partners
  - ☐ Enterprise
  - ☐ Government
  - ☐ Midmarket
  - ☐ Small Business
- Discount Band
- ☐ High
  - ☐ Low
  - ☐ Medium
  - ☐ None

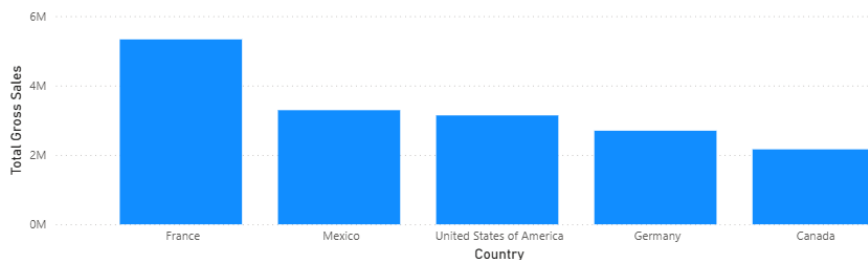
Total Gross Sales, Total Discounts and Total Profit



Total Gross Sales by Month Name



Total Gross Sales by Country



- Segment
- ☐ Channel Partners
  - ☐ Enterprise
  - ☒ Government
  - ☐ Midmarket
  - ☐ Small Business
- Discount Band
- ☐ High
  - ☐ Low
  - ☒ Medium
  - ☐ None

**Learning Outcomes:** The student should have the ability to

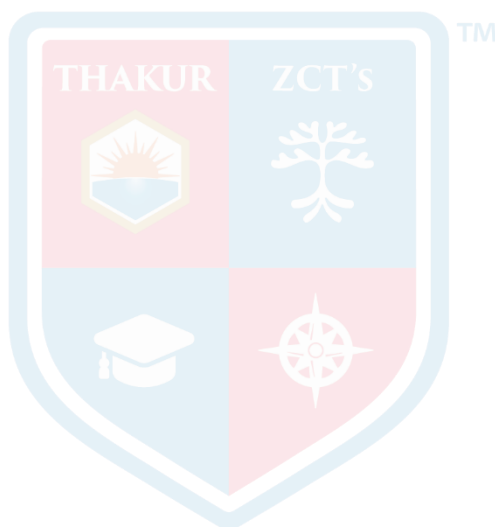
LO1: understand data connection and transformation in Power BI.

LO2: implement DAX functions to calculate key business metrics.

LO3: demonstrate an interactive sales dashboard with appropriate visual representations.

**Course Outcomes:** Upon completion of the course students will be able to apply business intelligence tools to visualize and analyze business data.

**Conclusion:**



For Faculty Use

| Correction Parameters | Formative Assessment [40%] | Timely completion of Practical [40%] | Attendance / Learning Attitude [20%] |  |
|-----------------------|----------------------------|--------------------------------------|--------------------------------------|--|
| Marks Obtained        |                            |                                      |                                      |  |